

Capitolo 1 - Intro

Status Done

Sistema operativo:

Insieme di programmi (software) che gestisce l'hardware. Fornisce una piattaforma ai programmi applicativi e agisce da intermediario fra l'utente e la struttura fisica del calcolatore.

Un sistema di elaborazione si può suddividere in quattro componenti:

- Hardware:
 - Composto dall'unità centrale d'elaborazione (CPU), dalla memoria e dai dispositivi di I/O.
 - Fornisce al sistema le risorse elaborative fondamentali.
- Programmi applicativi
 - Definiscono il modo in cui si usano le risorse per la risoluzione dei problemi computazionali degli utenti.
- Sistema operativo
 - Controlla l'hardware e ne coordina l'utilizzo da parte dei programmi applicativi per gli utenti
- Utente

Definizione più comune di SO:

Il SO è il solo programma che funziona sempre nel calcolatore, generalmente chiamato **kernel**.

Oltre al kernel troviamo due tipi di programmi:

- **programmi di sistema**, associato al SO;
- **programmi applicativi**, non correlati al funzionamento del sistema.

Organizzazione di un sistema elaborativo

Un moderno calcolatore è composto da una o più cpu e da un certo numero di controllori di dispositivi connessi attraverso un canale di comunicazione comune (*bus*) che permette l'accesso alla memoria condivisa del sistema. Ciascuno di questi controllori si occupa di particolare tipo di dispositivo fisico e può gestire una o più unità a esso connesse.

Un controllore di dispositivo dispone di una propria memoria interna (*buffer*) e di un insieme di registri specializzati.

Il controllore è responsabile del trasferimento dei dati tra i dispositivi periferici a esso connessi e la propria memoria interna.

I SO possiedono per ogni controllore di dispositivo un driver che gestisce le specificità del controllore e funge da interfaccia uniforme con il resto del sistema.

La cpu e i controllori possono eseguire operazioni in parallelo, competendo per i cicli di memoria. Per garantire un accesso ordinato alla memoria condivisa un controllore della memoria sincronizza gli accessi.

Interruzioni

L'hardware può generare un'interruzione in qualsiasi momento inviando un segnale alla cpu, solitamente tramite il bus di sistema.

Bus di sistema:

Principale percorso di comunicazione tra i componenti fondamentali.

Le interruzioni sono utilizzate per vari scopi e sono una componente chiave nell'iterazione tra i SO e l'hardware.

Quando la cpu riceve un segnale d'interruzione, interrompe l'elaborazione corrente e trasferisce l'esecuzione a una locazione fissa della memoria. Questa locazione contiene l'indirizzo iniziale della procedura di servizio dell'interruzione. Una volta completata l'esecuzione della procedura richiesta, la cpu riprende l'elaborazione che era stata interrotta.

Ogni interruzione deve causare il trasferimento del controllo all'appropriata procedura di servizio. Il modo più semplice è utilizzando una procedura generale che esamina le informazioni associate all'interruzione e invoca la procedura di gestione dello specifico evento. Ma la gestione delle interruzioni deve essere rapida in quanto si verificano frequentemente.

Perciò, è possibile utilizzare una **tabella di puntatori** alle specifiche procedure, l'attivazione delle procedure di servizio delle interruzioni così avvengono in modo diretto senza procedure intermedie. Generalmente, questa tabella è memorizzata nella memoria agli indirizzi più bassi. L'accesso a questa sequenza di indirizzi (**vettore delle interruzioni**) avviene per mezzo di un indice, codificato nello stesso segnale d'interruzione in modo da fornire l'indirizzo della procedura di servizio relativa all'evento segnalato.

Se la procedura di gestione dell'interruzione richiede la modifica dello stato del processore, deve salvare esplicitamente lo stato corrente per poterlo ripristinare prima di restituire il controllo. Terminato il servizio dell'interruzione, l'indirizzo di ritorno salvato viene caricato nel program counter in modo da riprendere la computazione dove era stata interrotta.

Implementazione

La maggior parte delle cpu ha due linee di richiesta di interruzione:

- **non mascherabile** (*nonmaskable interrupt*) → per eventi come errori irreversibili di memoria;
- **mascherabile** (*maskable interrupt*) → può essere disattivata dalla cpu prima dell'esecuzione di sequenze di istruzioni critiche che non devono essere interrotte, utilizzate dai controllori dei dispositivi per richiedere un servizio.

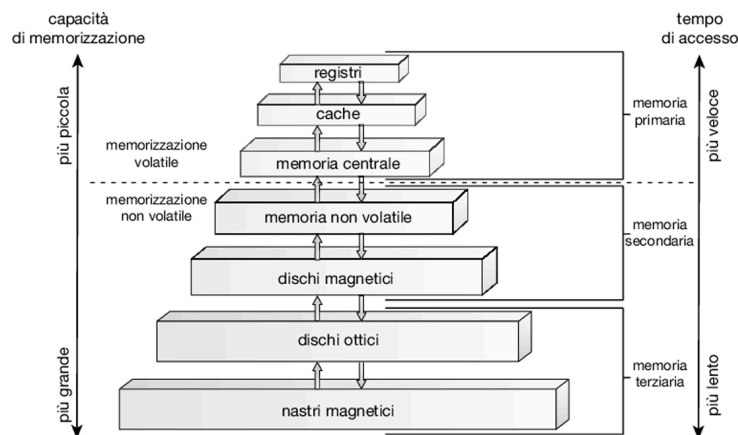
Il meccanismo di interruzione basato sul vettore delle interruzioni serve ad evitare a un singolo gestore di dover cercare tra tutte le possibili fonti per determinare chi ha sollevato l'interruzione. Tuttavia, i computer hanno più dispositivi di quanti siano i campi presenti nel vettore. Per risolvere questo problema si utilizza il **concatenamento delle interruzioni** (*interrupt chaining*), ogni elemento nel vettore delle interruzioni punta alla testa di un elenco di gestori.

Quando viene sollevata un'interruzione, i gestori nell'elenco corrispondente vengono chiamati uno ad uno, finché non ne viene trovato uno in grado di servire la richiesta.

Il meccanismo di interrupt implementa anche un sistema di **livelli di priorità** delle interruzioni, in modo da consentire alla cpu di posticipare la gestione delle interruzioni a bassa priorità senza mascherare tutte le interruzioni e permettono a un'interruzione ad alta priorità di essere gestita prima delle altre.

Struttura della memoria

La cpu può caricare istruzioni esclusivamente dalla memoria, quindi tutti i programmi da eseguire devono esservi caricati, la maggior parte dei programmi vengono eseguiti da una memoria riscrivibile: la memoria principale (RAM - Random Access Memory)



Struttura di I/O

Un calcolatore general-purpose è composto da un insieme di dispositivi connessi mediante un bus comune.

L'I/O guidato dalle interruzioni è adatto al trasferimento di piccole quantità di dati, ma in caso di trasferimenti massicci genera un pesante sovraccarico (*overhead*). Per risolvere questo problema si utilizza la tecnica del **Direct Memory Access** (DMA). Una volta impostati i buffer, i puntatori e i contatori necessari al dispositivo di I/O, il controllore trasferisce un intero blocco di dati dal proprio buffer direttamente nella memoria centrale, o viceversa, senza intervento da parte della cpu. L'operazione richiede una sola interruzione per ogni blocco di dati trasferito, lasciando la cpu libera.

Architettura degli elaboratori

Sistemi monoprocesso

Anni fa la maggior parte dei sistemi utilizzava un solo processore contenente un'unica cpu con un unico nucleo di elaborazione.

Nucleo di elaborazione:

componente che esegue le istruzioni e utilizza registri per memorizzare i dati localmente.

La singola cpu con il suo nucleo di elaborazione è in grado di eseguire un insieme di istruzioni di natura generale. Questi sistemi possiedono altri processori specializzati per compiti particolari. Questi processori sono dotati di un insieme ristretto di istruzioni, e non eseguono processi utenti. Sono guidati dal sistema operativo, che invia loro informazioni sul compito da eseguire successivamente e ne controlla lo stato.

Sistemi multiprocessore

Un sistema di questo tipo dispone di due o più processori, ciascuno con una cpu dotata di una singola unità di calcolo, condividono il bus e talvolta il clock di sistema, la memoria e i dispositivi periferici. Il vantaggio principale dei sistemi multiprocessore è la maggiore capacità elaborativa (throughput), infatti è possibile svolgere un lavoro maggiore in meno tempo. Se più unità di lavoro collaborano nell'esecuzione di un compito, è necessario un certo lavoro supplementare (overhead) per garantire che tutti i componenti funzionino correttamente.

I sistemi più comuni utilizzano la **multielaborazione simmetrica** (*symmetric multiprocessing*), in cui ogni processore è abilitato all'esecuzione di tutte le operazioni del sistema, incluse le funzioni del sistema operativo e i processi utente.

Vantaggio (smp): molti processi sono eseguibili contemporaneamente senza causare un calo delle prestazioni.

Svantaggio (smp): poiché le unità di elaborazione sono separate, una potrebbe essere inattiva, mentre un'altra è sovraccarica, causando un'inefficienza che sarebbe evitabile se le unità di elaborazione condividessero alcune strutture dati.

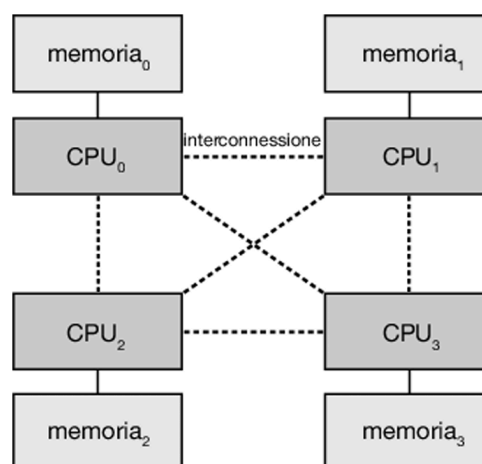
La definizione di multiprocessore si è evoluta e ora include i **sistemi multicore**, in cui più unità di calcolo risiedono su un singolo chip. Questi sistemi possono essere più efficienti rispetto a più chip dotati di una singola unità di calcolo, perché la comunicazione all'interno di un singolo è più veloce rispetto a quella tra un chip e un altro.

Un altro approccio, noto come **accesso non uniforme alla memoria (numa)**, è quello di fornire a ciascuna cpu la propria memoria locale accessibile per mezzo di un bus locale. Le cpu sono collegate da un'interconnessione di sistema condivisa, in modo che tutte le cpu condividano uno spazio di indirizzamento fisico.

Vantaggio (numa): quando una cpu accede alla sua memoria locale, non solo è veloce, ma non vi è alcun conflitto sull'interconnessione di sistema.

Svantaggio (numa): maggiore latenza quando una cpu deve accedere alla memoria remota attraverso l'interconnessione di sistema, con possibile peggioramento delle prestazioni.

Esempio: la cpu₀ non può accedere alla memoria locale della cpu₃ alla stessa velocità con cui può accedere alla propria memoria locale.



I SO possono minimizzare questo svantaggio del numa attraverso un attento scheduling della cpu e un'accurata gestione della memoria.

Cluster di elaboratori

Sono un altro tipo di sistemi multiprocessore, basati sull'uso congiunto di più cpu, ma differiscono per il fatto che sono composti di due o più calcolatori completi, detti **nodi**, collegati tra loro. Sistemi di questo tipo sono detti debolmente accoppiati (*loosely coupled*).

La definizione generale è che si tratta di calcolatori che condividono la memoria di massa, connessi per mezzo di una rete locale (lan), o da connessioni più veloci come InfiniBand.

Di solito si adotta il clustering per offrire un'elevata disponibilità, ciascun calcolatore esegue uno strato software di gestione del cluster, ogni nodo può tenere sotto controllo (attraverso lan) uno o più degli altri nodi. Se il nodo controllato si guasta, il calcolatore che lo sta supervisionando può prendere il controllo della sua memoria e riavviare le applicazioni che erano in esecuzione. Gli utenti e i client delle applicazioni notano solo una breve interruzione del servizio.

La capacità di continuare a offrire un servizio proporzionale alla quantità di hardware ancora in funzione è detta **degrado controllato** (*graceful degradation*). Alcuni sistemi si spingono oltre il degrado controllato e sono chiamati tolleranti ai guasti (*fault-*

tolerant), perché continuano a funzionare nonostante il guasto di un qualunque singolo componente. La tolleranza ai guasti necessita di un meccanismo per il riconoscimento del danno, la sua diagnosi, ed eventualmente, la correzione.

I cluster di elaboratori sono strutturabili in due modi:

- **cluster asimmetrici**: un calcolatore rimane nello stato di attesa attiva (*hot standby mode*) mentre l'altro esegue le applicazioni. Il calcolatore in *hot standby* monitora il server attivo e in caso di problemi, il calcolatore di controllo diventa il server attivo.
- **cluster simmetrici**: due o più calcolatori eseguono le applicazioni e allo stesso tempo si controllano reciprocamente; questo porta maggiore efficienza, in quanto vengono utilizzate meglio le risorse ma ciò richiede che siano disponibili più applicazioni da eseguire.

I cluster possono anche essere utilizzati per ottenere ambienti di elaborazione ad alte prestazioni (*high-performance computing*). Offrono molta più potenza elaborativa perché permettono l'esecuzione contemporanea di un'applicazione su tutti i computer del cluster. Tuttavia, le applicazioni devono essere scritte specificatamente in modo da trarre vantaggio del cluster sfruttando una tecnica chiamata **parallelizzazione** (*parallelization*).

Parallelizzazione:

suddividere il programma in componenti separate, eseguibili in parallelo su singoli computer all'interno del cluster. Queste applicazioni sono progettate in modo tale che, una volta che ogni nodo di elaborazione del cluster risolve la sua porzione di problema, tutti i risultati vengono combinati in un'unica soluzione finale.

Altre forme di clustering sono i cluster paralleli, permettono a più calcolatori di accedere agli stessi dati nella memoria condivisa, e i cluster di sistemi connessi attraverso reti geografiche (wan).

Attività del sistema operativo

Il SO è l'ambiente di esecuzione dei programmi. L'avviamento del sistema, richiede la presenza di un programma iniziale specifico, detto **programma di avviamento** (*bootstrap program*). Esso è memorizzato nel *firmware* (parte dell'hardware). La sua funzione è quella di inizializzare i vari componenti del sistema, come i registri della cpu, i controllori dei dispositivi e la memoria centrale. Il programma di avviamento deve caricare il sistema operativo e avviarne l'esecuzione, deve quindi individuare e caricare nella memoria il kernel del sistema operativo.

Una volta caricato e in esecuzione, il kernel può iniziare a offrire servizi al sistema e agli utenti. Alcuni servizi vengono forniti all'esterno del kernel da programmi di sistema caricati in memoria durante l'avviamento. Questi **processi di sistema**, o **demoni**, restano in esecuzione per tutto il tempo in cui è in esecuzione il kernel. (es. in Linux il primo processo di sistema è *systemd*, che avvia altri demoni).

Una volta completata questa fase il sistema è inizializzato e attende che si verifichi qualche evento. Un evento è quasi sempre segnalato da un'interruzione.

Un'altra forma di interruzione sono le **eccezioni** (*trap* o *exception*) ossia interruzioni generate dal software causate da un errore, o dalla richiesta di erogazione, da parte di un programma utente, di uno dei servizi del SO, realizzata mediante chiamata di sistema (*system call*).

Multiprogrammazione e multitasking

Tra le principali caratteristiche dei SO vi è la multiprogrammazione:

- permette agli utenti di eseguire più programmi nello stesso momento;
- consente di aumentare la percentuale di utilizzo della cpu, organizzando i programmi in modo tale da mantenerla in continua attività.

In un sistema multiprogrammato un programma in esecuzione è detto **processo**.

Il SO tiene contemporaneamente in memoria centrale diversi processi, ne sceglie uno e inizia l'esecuzione, ad un certo punto il processo potrebbe trovarsi nell'attesa di qualche evento, a questo punto il SO passa a un altro processo e lo esegue. Quando anche questo processo deve attendere, la cpu passa ad un altro processo ancora. Quando il primo processo ha terminato l'attesa, la cpu ne riprende l'esecuzione.

Il **multitasking** è un'estensione logica della multiprogrammazione, la cpu esegue più lavori commutando le loro esecuzioni con una frequenza tale da permettere a ciascun utente di usufruire di tempi di risposta rapidi.

Duplica modalità di funzionamento

Dal momento che il SO e i suoi utenti condividono le risorse hardware e software, un SO progettato correttamente deve assicurarci che un programma errato non possa causare il malfunzionamento di altri programmi. L'approccio adottato dalla maggior parte dei sistemi elaborativi è quello di fornire un supporto hardware che consenta la differenziazione fra le varie modalità di esecuzione. Sono necessarie almeno due diverse modalità:

- **modalità utente,**
- **modalità di sistema** (*modalità kernel o privilegiata*).

Per indicare quale sia la modalità attiva, la cpu è dotata di un bit (*bit di modalità*): kernel (0) o user (1), questo consente di stabilire se l'istruzione corrente si esegue per conto del SO o di un utente. Tuttavia, quando l'applicazione utente richiede un servizio al SO (tramite chiamata di sistema) per soddisfare la richiesta il sistema deve passare dalla modalità utente alla modalità di sistema.

All'avviamento, il bit è posto in modalità di sistema. Ogni volta che si verifica un'interruzione o un'eccezione, l'hardware passa dalla modalità utente a quella di sistema. Prima di passare il controllo al programma utente, il sistema ripristina la modalità utente riportando a 1 il valore del bit.

La duplice modalità di funzionamento consente la protezione del sistema operativo e degli altri utenti dagli errori di un utente. Questo livello di protezione si ottiene definendo come istruzioni privilegiate le istruzioni di macchina che possono causare danni allo stato del sistema.

Timer

Occorre assicurare che il SO mantenga il controllo della cpu, che consiste nell'impedire che un programma utente entri in un ciclo infinito o non richieda servizi del sistema senza più restituire il controllo al SO. Si può utilizzare un **timer programmabile** affinché invii un segnale d'interruzione alla cpu a intervalli di tempo specificati, che possono essere **fissi** o **variabili**. Solitamente si realizza mediante un clock a frequenza fissa e un contatore. Il SO assegna un valore al contatore, che si decrementa a ogni impulso, quando raggiunge il valore 0 si genera un segnale d'interruzione. Prima di restituire all'utente il controllo dell'esecuzione, il sistema assegna un valore al timer, se esso esaurisce questo intervallo genera un'interruzione che causa il trasferimento del controllo del SO. Esso poi può decidere se gestire l'interruzione come un errore fatale o concedere altro tempo al programma. Le istruzioni usate dal sistema per modificare il valore del timer si possono eseguire soltanto in modalità privilegiata.

Gestione delle risorse

Gestione dei processi

Un programma fa qualcosa soltanto se la cpu esegue le istruzioni che lo costituiscono. Consideriamo un processo come un lavoro d'elaborazione o un programma eseguito in un ambiente time-sharing.

Un processo necessita di alcune risorse per svolgere il proprio compito (es. tempo di cpu, memoria, file, I/O). Queste risorse si possono attribuire al processo al momento della sua creazione, o si possono assegnare durante l'esecuzione.

Un programma di per sé non è un processo d'elaborazione, ma un'**entità passiva**, come il contenuto di un file memorizzato in un disco, mentre un processo è un'**entità attiva**.

Un processo a singolo thread ha un contatore di programma che indica la successiva istruzione da eseguire, l'esecuzione di tale processi deve essere sequenziale: la cpu esegue le istruzioni del processo una dopo l'altra. Anche se due processi possono corrispondere allo stesso programma si considerano in ogni caso due sequenze d'esecuzione separate.

Un processo multithread possiede più contatori di programma, ognuno dei quali punta all'istruzione successiva da eseguire per un dato thread.

Un sistema è costituito di un gruppo di processi:

- alcuni eseguono **codice di sistema**;
- altri **codice utente**.

Tutti questi processi si possono eseguire in modo concorrente, avvicinandosi nell'uso di una singola unità di calcolo o in parallelo su diverse unità.

Il SO è responsabile delle seguenti attività connesse alla gestione dei processi:

- creazione e cancellazione dei processi utenti e di sistema;
- schedulazione di processi e thread sulle cpu;
- sospensione e ripristino dei processi;
- fornitura di meccanismi per la sincronizzazione dei processi;
- fornitura di meccanismi per la comunicazione tra processi.

Gestione della memoria

La memoria centrale è fondamentale per il funzionamento di un sistema elaborativo. È un vasto vettore di parole, ciascuna dotata di un proprio indirizzo.

È un archivio di dati velocemente accessibile, condiviso dalla cpu e dai dispositivi di I/O.

La cpu legge le istruzioni dalla memoria centrale durante il ciclo di prelievo delle istruzioni, oltre a leggere e scrivere i dati nella memoria centrale durante il ciclo d'accesso ai dati.

Generalmente la memoria centrale è l'unico dispositivo di memorizzazione a cui la cpu può far riferimento e accedere in modo diretto. Per eseguire un programma è necessario che questo sia associato a **indirizzi assoluti** e sia caricato nella memoria. Durante l'esecuzione del programma, esso accede alle proprie istruzioni e ai dati provenienti dalla memoria, generando indirizzi assoluti. Quando il programma termina, si rende disponibile il suo spazio di memoria, in modo da caricare ed eseguire il programma successivo.

Per migliorare l'utilizzo della cpu e la rapidità con la quale il calcolatore risponde i computer devono tenere molti programmi in memoria. Esistono diversi schemi di gestione della memoria, la scelta di un particolare schema dipende principalmente dal tipo di architettura hardware del sistema.

Il sistema operativo è responsabile:

- tenere traccia di quali parti di memoria sono attualmente usate e da chi;
- assegnare e revocare lo spazio di memoria secondo le necessità;
- decidere quali processi e dati debbano essere caricati in memoria centrale o trasferiti sulla memoria di massa.

Gestione dei file

Il SO associa i file a supporti fisici e vi accede tramite i dispositivi di memorizzazione delle informazioni. I calcolatori possono registrare le informazioni su mezzi fisici diversi, ciascuno ha caratteristiche proprie e una propria organizzazione fisica, ed è controllato da un dispositivo come un'unità a disco, avente anch'esso caratteristiche proprie. Queste proprietà comprendono velocità, capacità, rapidità nel trasferimento dei dati, metodi d'accesso (diretto o sequenziale).

Un file è una raccolta d'informazioni correlate definita dal suo creatore. Generalmente, i file rappresentano programmi, sia sorgente sia oggetto, e dati.

I file sono generalmente organizzati in directory. Se più utenti hanno accesso ai file, si potrebbe voler controllare chi ha la possibilità di accedervi e in che modo.

Il SO è responsabile:

- creazione e cancellazione dei file;
- creazione e cancellazione di directory;
- fornitura delle funzioni fondamentali per la gestione di file e directory;
- associazione dei file ai dispositivi di memoria secondaria;
- creazione di copie di riserva (*backup*) dei file su dispositivi di memorizzazione non volatili.

Gestione della memoria di massa

Il calcolatore deve disporre di una memoria secondaria a sostegno della memoria centrale (es. dischi magnetici, nvm). I programmi si servono del disco sia come sorgente sia come destinazione delle loro elaborazioni, perciò è fondamentale che la registrazione nella memoria secondaria sia gestita correttamente. Il sistema operativo è responsabile:

- montare (*mounting*) e smontare (*unmounting*) le unità di memoria;
- gestione dello spazio libero;
- assegnazione dello spazio;
- scheduling del disco;
- partizionamento,
- protezione.

Il frequente uso della memoria secondaria impone una sua gestione efficiente. Infatti, l'efficienza complessiva di un calcolatore può dipendere dalla velocità del sottosistema di gestione dei dischi. Vi sono svariati frangenti in cui tornano utili tali dispositivi di memorizzazione più lenti, meno costosi e più capienti della memoria secondaria, questi dispositivi vengono chiamati memoria terziaria.

Gestione della cache

Di norma le informazioni sono mantenute in un sistema di memoria come la memoria centrale; al momento del loro uso si copiano temporaneamente in un'unità più veloce, detta **cache**. Quando si deve accedere a una particolare informazione, si controlla se è già presente all'interno della cache; in tal caso si adopera direttamente la copia contenuta nella cache, altrimenti la si preleva dalla memoria centrale e la si copia nella cache, inoltre, i registri programmabili presenti all'interno della cpu, rappresentano per la

memoria centrale una cache ad alta velocità. Esistono anche cache che sono interamente gestite dall'hardware del sistema: per esempio la maggior parte dei sistemi è dotata di una cache per la memorizzazione delle istruzioni che presumibilmente saranno eseguite dopo l'istruzione corrente. Per motivi analoghi, la maggior parte dei sistemi è dotata di una o più cache di dati nella gerarchia della memoria. In una struttura gerarchica può accadere che gli stessi dati siano mantenuti contemporaneamente in diversi livelli del sistema di memorizzazione. In sistemi multiprocessore possono esistere più copie simultanee di un dato mantenute in cache differenti poiché le diverse unità di elaborazione possono operare in parallelo, è necessario che l'aggiornamento del valore di una qualsiasi delle copie di un dato si rifletta immediatamente in tutte le cache in cui il dato risiede. Questa situazione, nota come **coerenza della cache**, di solito si risolve a livello dell'hardware del sistema, quindi a un livello più basso di quello del sistema operativo. Essendoci la possibilità di accedere e modificare in modo concorrente queste repliche, è necessario fare in modo che ogni modifica a una qualunque replica si rifletta quanto prima sulle altre.

Gestione dell'I/O

Uno degli scopi di un sistema operativo è nascondere all'utente le peculiarità degli specifici dispositivi. Le caratteristiche di dispositivi di I/O sono nascoste dal sottosistema di I/O, che è composto da:

- un componente di gestione della memoria comprendente la gestione dei buffer di I/O, la gestione delle cache e la gestione delle aree di memoria per l'I/O asincrono (*spooling*);
- un'interfaccia generale per i driver dei dispositivi;
- i driver per gli specifici dispositivi.

Sicurezza e protezione

L'accesso ai dati deve essere disciplinato da regole, perciò vi sono meccanismi che assicurano che i file, i segmenti di memoria, la cpu e le altre risorse possano essere manipolati solo dai processi che abbiano ottenuto apposita autorizzazione dal SO.

Per protezione si intende ciascun meccanismo di controllo dell'accesso alle risorse possedute da un elaboratore, da parte di processi o utenti.

La protezione può migliorare l'affidabilità rilevando errori nascosti alle interfacce tra i componenti dei sottosistemi. Non proteggere una risorsa significa lasciare via libera al suo utilizzo da parte di utenti incompetenti o non autorizzati.

Pur essendo dotato di protezione adeguata, un sistema può rimanere esposto agli accessi abusivi, rischiando malfunzionamenti. È compito della sicurezza difendere il sistema da attacchi provenienti dall'interno o dall'esterno (es. virus e worm, attacchi DoS, ecc.). Per alcuni sistemi l'attività di prevenzione è considerata una funzione del sistema, altri delegano la difesa a regole operative o a programmi aggiuntivi.

Protezione e sicurezza presuppongono che il sistema sia in grado di distinguere tra tutti i propri utenti. Nella maggior parte dei sistemi operativi è disponibile un elenco di nomi degli utenti e dei loro identificatori (user id), ovvero id numerici che identificano univocamente l'utente. Questo id è associato a tutti i processi e i thread del soggetto in questione. In certe circostanze è preferibile distinguere tra gruppi di utenti, un utente può far parte di uno o più gruppi. L'identificatore di gruppo è incluso in tutti i processi e i thread a esso relativi.

Durante il normale utilizzo del sistema, è sufficiente un id utente e un id del gruppo. Tuttavia, gli utenti devono talvolta scalare i privilegi e ottenere permessi ausiliari per certe attività.

Virtualizzazione

La virtualizzazione è una tecnica che permette di estrarre l'hardware di un singolo computer in diversi ambienti di esecuzione, creando così l'illusione che ogni distinto ambiente sia in esecuzione sul proprio computer. Un utente di una macchina virtuale può passare da un sistema operativo ad un altro esattamente come può commutare tra processi in esecuzione contemporaneamente in un singolo sistema operativo.

In senso lato, la virtualizzazione appartiene a una tipologia di software che include anche l'emulazione. Questa tecnica viene utilizzata quando il tipo di cpu originale è diverso dal tipo di cpu di destinazione, lo stesso concetto può essere esteso per permettere a un intero SO scritto per una piattaforma di essere eseguito su piattaforme diverse.

L'emulazione ha un costo elevato, poiché ogni istruzione che può essere eseguita nativamente sul sistema sorgente deve essere tradotta nell'equivalente funzione da eseguire sul sistema di destinazione.

Con la virtualizzazione invece un sistema operativo compilato per una particolare architettura viene eseguita all'interno di un altro sistema operativo progettato per la stessa cpu, di fondamentale importanza è il VMM (gestore della macchina virtuale). Un VMM permette all'utente di installare su un computer portatile o desktop più sistemi operativi e di eseguire applicazioni scritte per sistemi operativi diversi da quello installato nativamente.

Sistemi distribuiti

Per sistema distribuito si intende un insieme di elaboratori fisicamente separati e con caratteristiche spesso eterogenee, interconnessi da una rete per consentire agli utenti l'accesso alle varie risorse dei singoli sistemi.

Una rete si può considerare come un canale di comunicazione tra due o più sistemi, differiscono per i protocolli usati, per le distanze tra i nodi e per il mezzo attraverso il quale avviene la comunicazione. Il protocollo di comunicazione più diffuso è il tcp/ip.

Le reti si classificano secondo le distanze tra i loro nodi:

- rete locale (lan), collega nodi all'interno della stessa stanza;
- rete geografica (wan), gruppi di edifici, città, ecc;
- rete personale (pan), es. dispositivi bluetooth.

Un sistema operativo di rete è dotato di caratteristiche quali la condivisione dei file attraverso la rete e di un modello di comunicazione per i processi attivi su elaboratori diversi, che possono così scambiarsi messaggi. Un sistema operativo distribuito offre un ambiente meno autonomo: la stretta comunicazione tra i diversi elaboratori tende a dare l'impressione che vi sia un solo sistema operativo che controlla la rete.

Strutture dati del kernel

Liste, stack e code

Un array è una semplice struttura dati in cui ogni elemento è accessibile direttamente. Le più importanti strutture dati dopo gli array sono probabilmente le liste, i dati presenti in una lista sono accessibili solo in un particolare ordine. Il più comune metodo per implementare questo tipo di struttura è la lista concatenata:

- in una lista semplicemente concatenata;
- in una lista doppiamente concatenata;
- in una lista circolare.

Le liste concatenate possono contenere dati di diversa dimensione e permettono di inserire e rimuovere elementi in maniera semplice. Un potenziale svantaggio dell'utilizzo di liste è che il tempo per prelevare un fissato elemento di una lista di dimensione N è lineare, $O(n)$.

Uno stack è una struttura dati dotata di un ordine sequenziale che utilizza una politica di tipo LIFO, le operazioni di inserimento e cancellazione di uno stack si chiamano push e pop.

Una coda è una struttura dati ordinata sequenzialmente che adotta una politica di accesso di tipo FIFO, le code sono anche piuttosto comuni nei sistemi operativi, per esempio nel caso di documenti inviati a una stampante.

Alberi

Un albero è una struttura dati utilizzabile per rappresentare i dati in maniera gerarchica. In un generico albero un padre può avere un numero illimitato di figli, mentre in un albero binario un padre ha al massimo due figli.

In un albero di ricerca binario si deve soddisfare un requisito: $figlio_sinistro \leq padre \leq figlio_destra$. Quando si cerca un elemento in un albero binario le prestazioni nel caso peggiore sono $O(n)$, per rimediare a questa situazione possiamo utilizzare un algoritmo per la creazione di un albero binario di ricerca bilanciato.

Funzioni e mappe hash

Una funzione hash riceve dati in input, realizza operazioni numeriche sui dati e restituisce un valore numerico. Mentre la ricerca di un elemento in una lista di dimensione n può richiedere fino a $O(n)$, utilizzando una funzione hash per il recupero di un dato si può arrivare anche a $O(1)$.

Un potenziale problema è il fatto che su input differenti si può produrre lo stesso valore di output, questo problema di collisione può essere risolto inserendo nella locazione una lista concatenata contenente tutti gli elementi con lo stesso valore hash.

Una funzione hash può essere usata per creare una tabella hash che associ coppie formate da una chiave e da un valore.

Ambienti d'elaborazione

Elaborazione Client-Server

Un'architettura di rete contemporanea realizza un sistema in cui alcuni server soddisfano le richieste dei sistemi client. Possiamo suddividere i sistemi server come:

- **server elaborativi**, forniscono un'interfaccia a cui i client possono inviare una richiesta, in risposta, il server esegue l'azione richiesta e restituisce i risultati al client.
- **file server**, offrono un'interfaccia di file system che consente al client creazione, aggiornamento, lettura e cancellazione dei file.

Elaborazione peer-to-peer

In questo modello non c'è la distinzione tra client e server, tutti i nodi all'interno del sistema sono su un piano di parità, ciascuno può fungere da client o server a seconda delle necessità.

Per entrare a far parte di un sistema peer-to-peer, un nodo deve prima unirsi alla rete degli altri sistemi e successivamente può iniziare a fornire e ottenere servizi agli altri nodi.

Due modalità generali per stabilire quali servizi siano disponibili:

- Quando il nodo si unisce alla rete, iscrive il proprio servizio in un **registro centralizzato di consultazione** della rete. Quando un nodo vuole ottenere un servizio, contatta il registro per verificare quali nodi lo forniscano.
- Schema alternativo: nessun servizio di consultazione centralizzato. Un nodo che operi come client deve accertare quale nodo fornisca il servizio desiderato, inoltrando la propria richiesta di servizio a tutti gli altri nodi della rete. I nodi che possono fornire tale servizio rispondono al nodo da cui è partita la richiesta.

Cloud computing

Il cloud computing è una tecnica che permette di fornire capacità elaborativa, storage e persino applicazioni come servizi di rete.

Diverse tipologie di cloud computing:

- cloud pubblico: un cloud disponibile attraverso internet;
- cloud privato: un cloud gestito da un'azienda per l'utilizzo al proprio interno;
- cloud ibrido: un cloud che comprende componenti pubbliche e private;
- SaaS: una o più applicazioni fruibili via internet;
- PaaS: un ambiente software predisposto per usi applicativi via Internet;
- IaaS: server o storage disponibili attraverso Internet.

Sistemi embedded real-time

In termini quantitativi, i computer facenti parte di altri sistemi attualmente costituiscono la tipologia predominante di elaboratore. Di solito hanno compiti molto precisi: i sistemi su cui sono installati sono spesso rudimentali, per cui offrono funzionalità limitate. Ciò che contraddistingue i sistemi embedded è la loro grande variabilità, che sfruttano applicazioni create appositamente per implementare una funzionalità. I sistemi embedded funzionano quasi sempre con sistemi operativi real time, per questa ragione sono spesso adoperati come dispositivi di controllo per applicazioni dedicate. I sensori trasmettono i dati al computer che deve analizzarli e prendere le misure adatte per il controllo del sistema.